2026 31st International Conference on Automation and Computing (ICAC) | 979-8-3195-3407-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICAC70325.2026.11712447

ROS-Free Augmented Reality Teleoperation of the Kinova Gen3 Robotic Arm Using Unity-Native Inverse Kinematics

Angela Kimie Matsuo
School of Engineering and Innovation
Aston University
Birmingham, United Kingdom
240293837@aston.ac.uk

Chao Liu
School of Engineering and Innovation
Aston University
Birmingham, United Kingdom
c.liu16@aston.ac.uk

Yu Jia
School of Engineering and Innovation
Aston University
Birmingham, United Kingdom
y.jia1@aston.ac.uk

Yuchun Xu
School of Engineering and Innovation
Aston University
Birmingham, United Kingdom
y.xu16@aston.ac.uk

***Abstract*—Most AR teleoperation systems for redundant robotic arms depend on ROS and Linux middleware or focus on usability evaluation without characterising technical performance. This paper presents a lightweight, ROS-free teleoperation framework for the Kinova Gen3 7-DoF arm, using a HoloLens 2 AR interface, a Unity-native Jacobian IK solver, and a Python UDP backend. The operator controls the arm through pinch gestures, with no physical controller or external middleware. Evaluated across 20 target positions, the system achieved 100% target acquisition, 2.97 cm mean holding accuracy, 0.72 s IK convergence, and 46.9 ms round-trip latency at 21 Hz. The Pearson correlation between latency and joint error was −0.023, confirming no measurable effect of communication delay on accuracy. This is, to the authors' knowledge, the first system for the Kinova Gen3 combining a ROS-free AR interface with a Unity-native IK solver and simultaneous quantitative evaluation of all four technical performance dimensions.**

Keywords—augmented reality, teleoperation, robotic arm, HoloLens 2, Kinova, inverse kinematics, digital twin, Industry 5.0

I. Introduction

The transition from Industry 4.0 to Industry 5.0 marks a shift in focus from the Internet of Things (IoT) and Big Data towards human-robot collaboration, artificial intelligence (AI), and sustainability. The goal is to improve human-machine collaboration rather than replacing humans with machines [1, 24]. The evaluation in this paper regards to quantitative technical measurements; no user study with human participants was conducted.

Many existing teleoperation systems rely on joysticks, 3D mice and 2D screens, which require operators to mentally map controller movements to robot motion through a flat display. Smith and Kennedy III [3] and Zhou et al. [15] report that this creates high cognitive demand and contributes to operator errors. AR interfaces address this by overlaying the robot's virtual model onto the operator's workspace, allowing gesture control without rotation mapping. Wu et al. [4] reported a 45% reduction in NASA-TLX scores comparing an MR interface against a 2D display and game controller, and Karpichev et al. [8] demonstrated the viability of real-time bidirectional digital twin communication for AR teleoperation.

A clear gap exists in the literature: most published AR teleoperation systems for the Kinova Gen3 rely on ROS and Linux infrastructure as the bridge between the AR application and the physical robot, and those that do evaluate performance focus primarily on usability outcomes such as NASA-TLX and SUS scores, without characterising communication latency, IK convergence time, positional accuracy, or digital twin fidelity in the same system. As far as the literature presented here indicates, no published work has reported all four of these technical measurements together within a single ROS-free AR teleoperation framework. Building on the developments reviewed above, this paper addresses this gap directly. It presents the design, implementation, and quantitative evaluation of a complete AR teleoperation framework for the Kinova Gen3, using the HoloLens 2, Unity, and a Python backend with a Jacobian IK solver. The system replaces traditional controllers with natural gesture interaction and removes the ROS and Linux dependency entirely. This work provides a technically validated and reproducible baseline for accessible AR teleoperation of redundant robotic manipulators, supporting the human-centred vision of Industry 5.0 [1, 24].

This paper makes four contributions: (1) a complete ROS-free AR teleoperation interface for the Kinova Gen3, deployed on the HoloLens 2 using Unity and MRTK, controlled through hand gestures alone; (2) a custom Jacobian IK solver with damped least squares ($\lambda = 5.0$) and null-space optimisation ($k_{ns} = 0.18$), running entirely within Unity on the HoloLens 2 without any external planning library; (3) a lightweight UDP pipeline from Unity to a Python backend that interfaces directly with the Kinova Kortex API, requiring no ROS node or Linux machine; and (4) a simultaneous quantitative evaluation of communication latency, positional accuracy, IK convergence time and per-joint digital twin fidelity within a single experimental session.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Authorized licensed use limited to: University of Washington Libraries. Downloaded on October 10,2026 at 16:34:36 UTC from IEEE Xplore. Restrictions apply.

While each component of this framework draws on established methods, their integration within the UWP constraints of the HoloLens 2, without ROS, without Linux, and without external IK libraries, constitutes an engineering contribution that has not been previously demonstrated for this platform. ROS cannot run natively on UWP, creating a hard infrastructure dependency that limits portability and reproducibility. The present framework removes this dependency entirely, making the system deployable on any Windows host. Beyond the architectural contribution, the simultaneous quantitative characterisation of all four technical performance dimensions within a single ROS-free system represents a reduction in the barrier for AR teleoperation in environments where Linux infrastructure is not available.

II. Related Work

Several recent systems demonstrate that AR teleoperation for the Kinova Gen3 is feasible. Smith and Kennedy III [3] deployed a HoloLens 2 Unity interface where operators control the arm by moving a virtual end-effector surrogate through pinch gestures, comparing it against the 3D SpaceMouse in a 15-participant study. The AR interface achieved higher usability scores, lower perceived task load and higher recommendation likelihood, with the biggest advantage on tasks requiring end-effector reorientation. Their system uses ROS# to communicate with a Linux machine running ROS Noetic. Wu et al. [4] proposed an MR approach where operators move digital twins of target objects to specify pick-and-place tasks, reporting 45% lower NASA-TLX scores and 31% higher SUS compared to a 2D display and game controller across 30 participants, also over a ROS-TCP setup. Meng et al. [5] implemented a Virtual Reality (VR) scene interaction system for the Kinova Gen3, finding shorter task completion times and lower mental workload than direct end-effector control.

Wang et al. [6] demonstrated gesture-driven AR control of a Franka Emika Panda arm with positioning errors below 0.35 cm, using marker calibration and a ROS backend. Lin et al. [7] found that AR visual cues reduced task completion time by roughly 32% compared to haptic feedback alone in Kinova Gen3 manipulation tasks. Karpichev et al. [8] proposed a ROS-Docker framework for bidirectional digital twin synchronisation with the Kinova Gen3, confirming that real-time synchronisation is technically viable but maintaining the ROS dependency. Joseph and Hu [17] implemented a gaze-driven shared autonomy framework for the Kinova Gen3 using a VR headset, finding that grasping assistance significantly reduced NASA-TLX physical demand and frustration scores, though task completion times were longer due to occasional grasp generation failures. Malagalage Don et al. [14] showed that AR-collected demonstrations produce smoother, jerk-free trajectories compared to joystick and kinesthetic teaching methods, resulting in higher machine learning policy success rates.

Open-source repositories such as frank-Regal [23] demonstrate community interest in AR teleoperation for the Kinova Gen3, providing a HoloLens 2 Unity interface for direct robot control. However, neither this nor comparable open-source implementations provide formal quantitative evaluation of positioning accuracy, latency or digital twin fidelity, leaving the performance of such systems uncharacterised in the literature.

None of the above systems implements a custom Jacobian IK solver within the HoloLens 2 Unity application, and none removes the ROS dependency entirely. Rice et al. [12] note that Jacobian methods remain the most practical choice for real-time digital twin applications in game engines such as Unity, since ROS-dependent analytical solvers like IKFast cannot be deployed on platforms where ROS is unavailable, reinforcing the case for a native Unity IK implementation. No published work reviewed here provides a combined quantitative evaluation of end-effector positional accuracy, IK convergence time, per-joint digital twin fidelity and communication latency within the same system and session. It should be noted that the evaluation reported in this paper is limited to quantitative technical measurements; no user study with human participants was conducted.

III. System Architecture

The system has three layers. The User Interaction Layer runs on the HoloLens 2 in Unity 2022.3.27f1 LTS [21], with MRTK [19] providing hand tracking. It captures pinch gestures, computes Cartesian poses through a Jacobian IK solver, and renders the digital twin. The Control and Communication Layer runs on the host computer, receiving joint angle commands over UDP at 20 Hz and forwarding them to the robot through the Kortex API [20], with no ROS dependency. Joint states are logged to CSV at 5 Hz but not transmitted back to Unity. The Physical Robot Layer is the Kinova Gen3 with a Robotiq 2F-85 gripper [22], connected over Wi-Fi. UDP is used over TCP to prioritise low latency, following Lim et al. [13], who identify it as the preferred protocol for loss-tolerant cyber-physical systems.

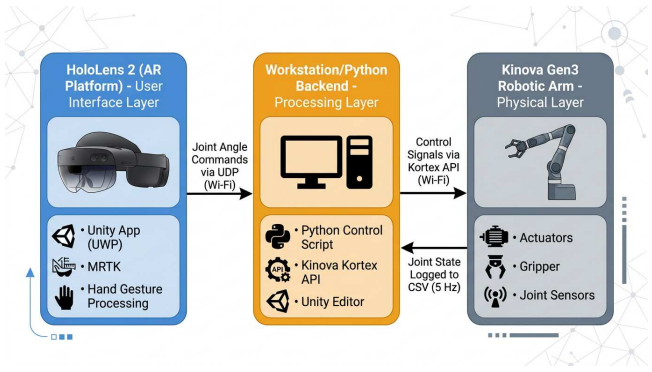


Fig. 1. System architecture of the AR-based Kinova teleoperation framework

The system is built across three layers. The User Interaction Layer runs on the HoloLens 2, developed in Unity 2022.3.27f1 Long-Term Support (LTS) [21] targeting UWP, with the Mixed Reality Toolkit (MRTK) [19] providing hand tracking and

Authorized licensed use limited to: University of Washington Libraries. Downloaded on October 10,2026 at 16:34:36 UTC from IEEE Xplore. Restrictions apply.

spatial interaction. It captures hand gestures, computes desired Cartesian poses through a Jacobian IK solver, and renders the digital twin in the operator's workspace. The Control and Communication Layer runs on the host computer, receiving joint angle commands from Unity over UDP at 20 Hz, applying smoothing, and forwarding them to the physical robot through the Kinova Kortex API [20], with no ROS node or Linux dependency. Joint positions and end-effector pose are read from the robot at 5 Hz and logged to CSV for subsequent analysis, but are not transmitted back to Unity during operation. The digital twin always reflects the configuration commanded by the IK solver rather than the physical robot's measured state. The Physical Robot Layer is the Kinova Gen3 arm with a Robotiq 2F-85 gripper [22], which executes the joint angle commands received through the Kortex API and connects to the host computer over the local Wi-Fi network.

IV. Inverse Kinematics

The KinovaIKController C# script running on the HoloLens 2 handles all IK computation, joint limit enforcement, smoothing, and hold mode. The Python backend applies a second smoothing stage and forwards commands to the robot, it performs no kinematic reasoning of its own.

The KinovaIKController runs within Unity's FixedUpdate loop and on each frame constructs a 3x7 Jacobian matrix J, where each column is the cross product of the joint's rotation axis and the vector from that joint to the end-effector. For joint i, the k-th row of column i is defined as:

$$J_s ki = (\hat{a}_i \times r_i)_{k,} \quad k \in \{x, y, z\}, i = 0, \dots, 6 \qquad (1)$$

Where â_i is the unit rotation axis of joint i, and r_i = p_ee − p_i is the vector from joint i to the end-effector. Joint rotation axes are pre-calibrated as alternating â_i = (0, 1, 0) and â_i = (−1, 0, 0) for even and odd joints respectively, reflecting the physical mounting orientation of the Kinova Gen3 arm in the laboratory, where the base is rotated 180° around the Y axis relative to the default URDF coordinate frame.

The joint velocity increment is computed via the damped pseudo-inverse:

$$\Delta\theta = J^T(JJ^T + \lambda I)^{-1}e \qquad (2)$$

Where e = p_target − p_ee is the Cartesian error vector, λ = 5.0 is the damping factor, and I is the 3x3 identity matrix. The damping term prevents large joint velocities near singular configurations.

A null-space projection is added to bias the redundant degrees of freedom towards a preferred joint configuration without affecting the primary end-effector task:

$$\Delta\theta_{ns} = k_ns(I - J^T(JJ^T + \lambda I)^{-1}J)(\theta_{pref} - \theta) \qquad (3)$$

Where k_ns = 0.18 is the null-space gain and θ_{pref} is the preferred joint configuration initialised at application startup.

The total joint update is scaled by gain g = 0.05 and passed through a per-frame low-pass filter with Unity-side smoothing coefficient α_U = 0.92 (filterSmoothing). Each joint update is additionally clamped to maxStepDeg = 0.3° per frame and constrained within joint mechanical limits. Before forwarding commands to the robot, the Python backend applies a second stage of exponential smoothing with α_P = 0.80, weighting the previous filtered value at 80% and the new incoming value at 20%. Two gating conditions control each transmission: the maximum absolute change across all seven joints must exceed MIN_DELTA = 0.4°, and the transmission rate must not exceed SEND_HZ = 20 Hz. Hold mode activates when the end-effector is within 30 mm of the target, locking the ArticulationBody drives at their current positions.

The contribution of the IK solver is the implementation entirely within a Unity C# script running on the HoloLens 2 UWP environment, a platform where ROS-dependent solvers such as IKFast cannot be deployed.

V. Experimental Setup and Evaluation Metrics

The tests were conducted in an Aston University robotics laboratory, designed for safe operation of collaborative robotic arms. The Kinova Gen3 is mounted vertically on a fixed base in the centre of the workspace, over a table positioned directly in front of it. The table is kept clear during trials, and an emergency stop button is within immediate reach of the operator at all times.

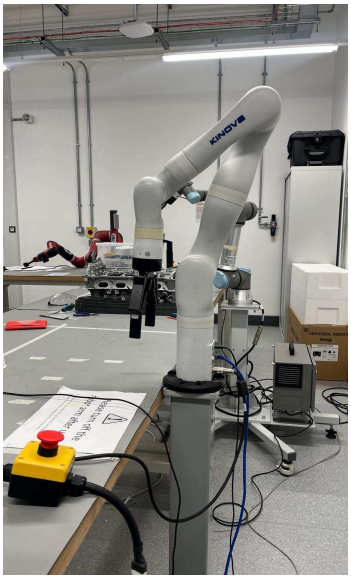


Fig. 2. Experimental setup at Aston University robotics laboratory

The arm connects to the host computer over Wi-Fi at a fixed IP address. At the start of each session, the operator visually aligns the holographic model over the physical arm. The evaluation consists of a single continuous session where the operator repositioned the holographic target cube across 20 positions covering close, mid-range and far workspace locations [2, 9]. Each target was held for 8 to 15 seconds. The session lasted

Authorized licensed use limited to: University of Washington Libraries. Downloaded on October 10,2026 at 16:34:36 UTC from IEEE Xplore. Restrictions apply.

approximately 196 seconds, producing 3,373 IK log samples and 841 comparison log samples.

The system is evaluated using four metrics, all calculated from the CSV files recorded during the session. The first is communication latency: how long each command cycle takes, from Unity sending a packet to the Python backend finishing the robot command and reading the joint state back, split into the time to send the command and the time to read the angles. The second is positional accuracy: the distance in centimetres between where the IK solver computes the end-effector position and where the target is, measured separately during movement and during holding. The third is task completion time, measured as IK convergence time: how long it takes from the moment the operator drops the cube to the moment the end-effector first reaches the 30 mm hold zone. The fourth is digital twin fidelity: the difference in degrees between the joint angles commanded and the angles the physical robot actually reached, measured at each joint separately at 5 Hz throughout the session. Shi et al. [9] note that the ForeArm joint is the most prone to tracking error in 7-DoF arms due to its position in the kinematic chain, making per-joint analysis particularly relevant for this platform.

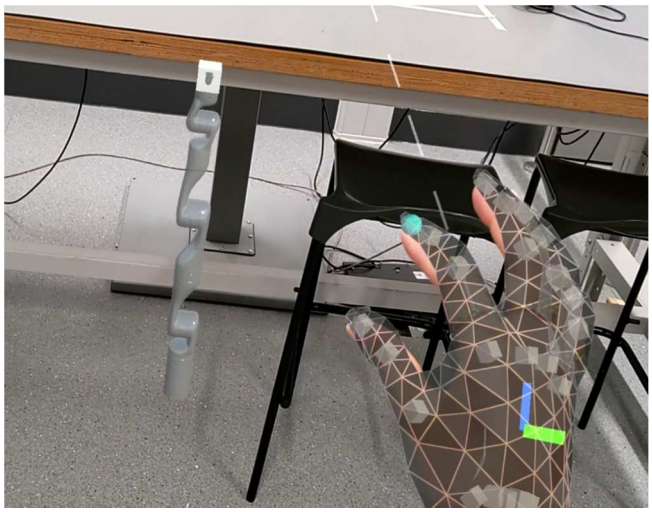


Fig. 3. Operator's view through the HoloLens 2 showing the MRTK hand mesh overlay and the holographic target cube during a pinch gesture interaction

VI. Results

A. Communication latency

Over 231 recorded cycles, send command latency averaged 21.7 ms (std 17.4 ms) and read angles latency averaged 25.3 ms (std 2.8 ms). The round-trip was 46.9 ms mean and 48.3 ms median, giving an effective rate of around 21 Hz. The 95th percentile was 54.0 ms and the 99th was 65.6 ms. One outlier at 309.8 ms appeared at the very start of the session, caused by connection initialisation on the first packet. With that point aside, the spread between p50 and p95 is only 5.7 ms, indicating low jitter. The Pearson correlation between round-trip latency and mean joint error was −0.023, confirming that communication delay had no measurable effect on tracking accuracy. When a packet arrived late, the robot just held its last position until the next one came through, so occasional delays never produced lasting tracking error [13].

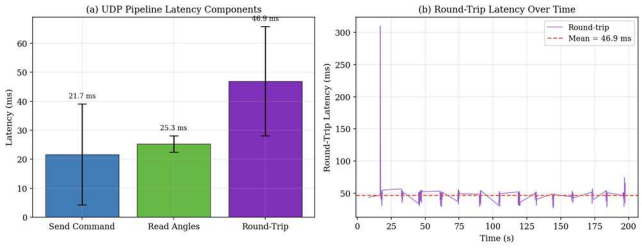


Fig. 4. UDP pipeline latency components and round-trip latency over time

B. Positional accuracy

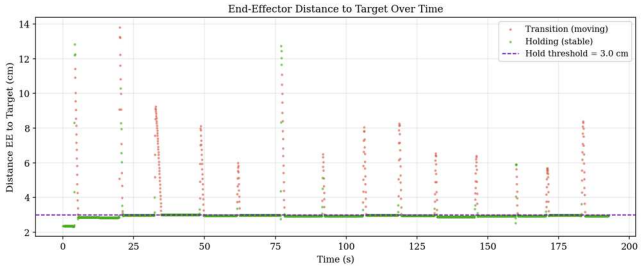
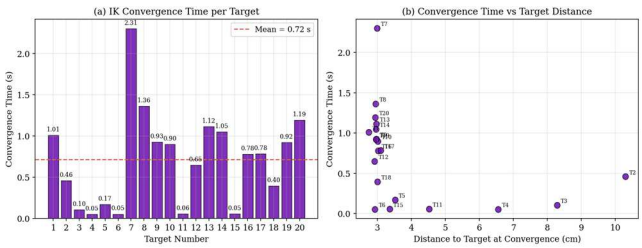


Fig. 5. End-effector distance to target over time, holding (green) and transition (red) samples

The end-effector reached the 30 mm hold threshold on all 20 targets without failure, a 100% acquisition rate. During the holding phase (3,117 of 3,373 samples), the mean distance was 2.97 cm. The overall mean across all samples including transitions was 3.20 cm, and the peak distance of 13.82 cm occurred during the largest workspace displacement of the session. For comparison, Wang et al. [6] report errors below 0.35 cm in their gesture-driven AR system for a Franka Emika Panda arm, though their system uses marker-based spatial calibration which provides much finer registration than the visual alignment used here.

C. IK convergence time

Mean convergence time was 0.72 seconds across 20 targets, ranging from 0.05 to 2.31 seconds. Four targets converged in under 0.1 seconds, as the operator placed the cube close to the end-effector's current position. The 7th target took the longest at 2.31 seconds, corresponding to the largest joint displacement of the session. The scatter plot of convergence time versus end-effector distance at convergence shows no clear relationship between the two variables, confirming that convergence time is determined by the joint-space path the arm must take rather than the Cartesian distance to the target [2, 9].



Authorized licensed use limited to: University of Washington Libraries. Downloaded on October 10,2026 at 16:34:36 UTC from IEEE Xplore. Restrictions apply.

Fig. 6. IK convergence time per target (mean = 0.72 s) and convergence time versus distance at convergence.

D. Digital twin fidelity

Per-joint MAE analysis across 841 samples showed a clear split. Four joints tracked the commanded angles very closely: Bracelet (0.03°), Wrist1 (0.10°), Shoulder (0.14°) and HalfArm2 (0.16°). Three mid-body joints showed meaningfully higher errors: ForeArm (MAE = 1.95°, peak = 19.72°), HalfArm1 (MAE = 1.46°, peak = 14.16°) and Wrist2 (MAE = 1.42°, peak = 10.90°). The overall mean across all seven joints was 0.75°, but the per-joint breakdown in Table I gives a clearer picture. Error spikes lined up exactly with transition windows between targets and returned to near-zero during hold phases, confirming a transition-phase effect rather than a persistent offset.

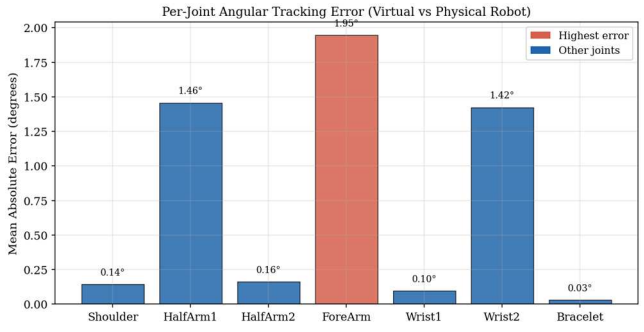


Fig. 7. Per-joint mean absolute error between commanded and measured angles (overall mean = 0.75°).

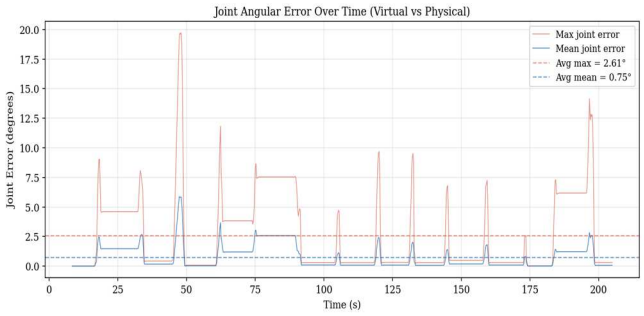


Fig. 8. Joint angular error over time showing mean (blue) and maximum (red) across all seven joints. Error spikes align with transition windows between targets and return to near-zero during hold phases (avg mean = 0.75°, avg max = 2.61°).

TABLE I. PER-JOINT MEAN ABSOLUTE ERROR BETWEEN COMMANDED AND MEASURED ANGLES (N = 841 SAMPLES), ORDERED FROM BASE TO END-EFFECTOR.

Joint	MAE	Peak Error	Group
Shoulder	0.14°	1.24°	Low error
HalfArm1	1.46°	14.16°	High error
HalfArm2	0.16°	1.50°	Low error
ForeArm	1.95°	19.72°	High error
Wrist1	0.10°	1.49°	Low error
Wrist2	1.42°	10.90°	High error
Bracelet	0.03°	0.28°	Low error

VII. DISCUSSION

The elevated errors at ForeArm, HalfArm1 and Wrist2 arise because the IK solver and the physical robot may resolve the same end-effector target using different configurations from the redundant solution set, and those differences accumulate at the mid-body joints most involved in that resolution. Shi et al. [9] identify the ForeArm axis as the most exposed to this behaviour in 7-DoF manipulators of this kinematic class. Since errors are transient and resolve within the convergence window on every target, they do not prevent task completion. Li et al. [10] report the same pattern in a comparable unidirectional digital twin pipeline, confirming this is expected behaviour. When a packet arrived late, the robot just held its last position until the next one came through, so occasional delays never produced lasting tracking error [13].

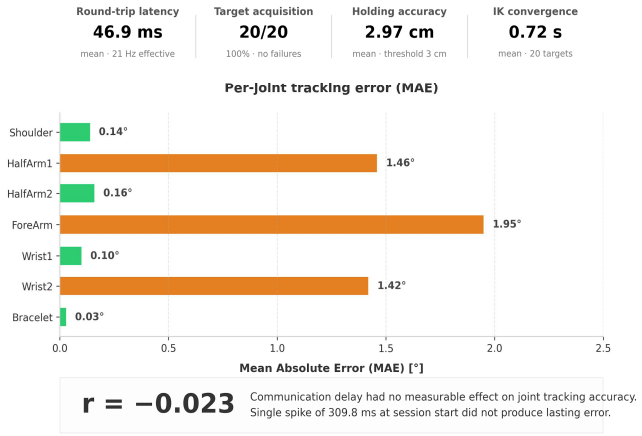


Fig. 9. System performance summary, key results of the four evaluation dimensions.

The 2.97 cm mean holding accuracy falls within the range reported for comparable gesture-driven AR teleoperation systems [6], and is suitable for reach-and-hold tasks in open workspaces. It would not meet the sub-millimetre requirements of surgical robotics applications [18]. The unidirectional pipeline is the most significant architectural limitation: the digital twin shows what was commanded, not what the robot actually did, so the holographic overlay can briefly diverge from the physical arm during movement. A return UDP channel would close this loop without requiring any structural change to the HoloLens 2 application. The position-only 3x7 Jacobian leaves end-effector orientation uncontrolled, ruling out insertion or precision grasping tasks; extending to a 6x7 full-pose Jacobian is the most direct improvement. The architecture is also compatible with learning-from-demonstration pipelines, since AR-collected demonstrations have been shown to produce higher-quality machine learning policies than kinesthetic teaching [14].

Authorized licensed use limited to: University of Washington Libraries. Downloaded on October 10,2026 at 16:34:36 UTC from IEEE Xplore. Restrictions apply.

Karpichev et al. [11] argue that XR-mediated human-in-the-loop architectures represent one of the most promising directions for Industry 5.0 human-robot collaboration. The present system supports this vision by making AR teleoperation deployable on any standard host computer without specialist middleware, directly addressing the accessibility goals of Industry 5.0 [1, 24].

VIII. Conclusion

This paper presented a ROS-free AR teleoperation framework for the Kinova Gen3 arm, addressing a gap in existing work where AR teleoperation systems for this platform either depend on ROS and Linux middleware or report usability outcomes without characterising technical performance. The results show that a HoloLens 2 AR interface, combined with a Unity-native Jacobian IK solver and a lightweight Python UDP backend, can achieve stable reach-and-hold teleoperation without any external middleware. The end-effector reached all 20 targets without failure, holding within 2.97 cm on average, with IK convergence averaging 0.72 seconds and round-trip latency averaging 46.9 ms at 21 Hz. The absence of correlation between latency and joint error ($r = -0.023$) confirms that the UDP pipeline is sufficient for this class of task. The per-joint analysis reveals a bimodal fidelity distribution consistent with 7-DoF redundancy behaviour, where four joints track within 0.2° and three mid-body joints show transient errors during transitions that resolve fully at each hold phase.

Three architectural limitations bound the scope of this work and are addressed explicitly as directions for future research. First, the digital twin pipeline is unidirectional: the holographic overlay reflects commanded rather than measured state, so brief divergence between the virtual and physical arm can occur during transitions. A return UDP channel would address this without structural changes to the HoloLens 2 application. Second, the position-only 3x7 Jacobian provides no orientation control of the end-effector, limiting the system to reach-and-hold tasks and ruling out insertion or precision grasping. Extending to a 6x7 full-pose Jacobian is the most direct path forward. Third, no user study was conducted, so the workload and usability characteristics of the interface remain unquantified. A controlled study with NASA-TLX and SUS measures comparing this system against a SpaceMouse or game controller baseline would provide the human factors evidence this paper does not offer.

Together, the results establish a technically validated and reproducible baseline for accessible AR teleoperation of redundant robotic manipulators, and confirm that removing the ROS dependency does not compromise the core teleoperation capability for reach-and-hold tasks. This framework directly supports the inclusive and human-centred goals of Industry 5.0 [1, 24].

References

[1] Lou, S., Hu, Z., Zhang, Y., Feng, Y., Zhou, M. and Lv, C. (2024) 'Human-cyber-physical system for Industry 5.0', *IEEE Transactions on Automation Science and Engineering*, pp. 1–18.

[2] Sepehri, A. and Moghaddam, A.M. (2021) 'A motion planning algorithm for redundant manipulators', *IEEE Access*, 9, pp. 26059–26072.

[3] Smith, A. and Kennedy III, M. (2025) 'An augmented reality interface for teleoperating robot manipulators', *arXiv*:2409.18394.

[4] Wu, Y., Zhao, B. and Li, Q. (2025) 'The teleoperation of robot arms by interacting with an object's digital twin in a mixed reality environment', *Applied Sciences*, 15, 3549.

[5] Meng, L., Liu, J., Chai, W., Wang, J. and Meng, M.Q.-H. (2023) 'Virtual reality robot teleoperation via human-scene interaction', *Procedia Computer Science*, 226, pp. 141–148.

[6] Wang, X., Guo, S., Xu, Z., Zhang, Z., Sun, Z. and Xu, Y. (2024) 'A robotic teleoperation system enhanced by augmented reality', *Cyborg and Bionic Systems*, 5, Article 0098.

[7] Lin, T.-C., Krishnan, A.U. and Li, Z. (2022) 'Comparison of haptic and augmented reality visual cues for assisting tele-manipulation', *ICRA 2022*, pp. 9309–9316.

[8] Karpichev, Y., Chick Zaouali, M., Charter, T. and Najjaran, H. (2024) 'A deployable and scalable ROS-Docker framework for multi-platform digital twin applications', *preprint submitted to IEEE*.

[9] Shi, X., Guo, Y., Chen, X., Chen, Z. and Yang, Z. (2021) 'Kinematics and singularity analysis of a 7-DOF redundant manipulator', *Sensors*, 21(21), p. 7257.

[10] Li, H., He, X., Wu, Y., Liu, G., Wang, H., Wen, X. and Li, L. (2026) 'Digital twin and AI-driven robotic embodied control system', *Robotics and Computer-Integrated Manufacturing*, 98, p. 103138.

[11] Karpichev, Y., Charter, T., Hong, J., Soufi Enayati, A.M., Honari, H., Ghafarian Tamizi, M. and Najjaran, H. (2024) 'Extended reality for enhanced human-robot collaboration: a human-in-the-loop approach', *RO-MAN 2024*, pp. 1991–1998.

[12] Rice, S., Azab, A. and Saad, S. (2024) 'Fusion IK: solving inverse kinematics using a hybridized deep learning and evolutionary approach', *Manufacturing Letters*, 41, pp. 9–18.

[13] Lim, K.Y.H., Zheng, P. and Chen, C.H. (2020) 'A state-of-the-art survey of Digital Twin', *Journal of Manufacturing Science and Engineering*, 142(6), p. 061011.

[14] Malagalage Don, T.T., Khayyam, H. and Asadi, E. (2024) 'Enhancing robotic manipulation: AR-powered data collection for learning from demonstration', *preprint*.

[15] Zhou, Q., Chacon, A., Pan, J. and Johal, W. (2025) 'Assisting MoCap-based teleoperation using augmented reality visualisations', *arXiv*:2501.05153.

[16] Platt, J. and Ricks, K. (2022) 'Comparative analysis of ROS-Unity3D and ROS-Gazebo', *Journal of Intelligent and Robotic Systems*, 106(4), p. 80.

[17] Joseph, K. and Hu, Y. (2025) 'Gaze to Grasp: Shared Autonomy in VR Robot Teleoperation', in 2025 34th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN), Eindhoven, The Netherlands, 25–29 August, pp. 530–536. DOI: 10.1109/RO-MAN63969.2025.11217715

[18] Peng, H., Yang, X., Su, Y.-H. and Hannaford, B. (2020) 'Real-time data driven precision estimator for RAVEN-II surgical robot end-effector position', *ICRA 2020*, pp. 9020–9026.

[19] Microsoft (2024) *MRTK Documentation: Developing for HoloLens 2*. Redmond: Microsoft Corporation.

[20] Kinova (2023) *Kortex API User Guide: Python Integration for Gen3 Robotic Arms*. Montreal: Kinova Robotics.

[21] Unity Technologies (2024) *Unity Manual: Version 2022.3 LTS*. San Francisco: Unity Technologies.

[22] Kinovarobotics (2024) *ros_kortex: kortex_description/grippers/robotiq_2f_85*. GitHub.

[23] frank-Regal (n.d.) *kinova_ar_teleop: Control Kinova Gen3 robots with HoloLens 2 augmented reality headset*. GitHub. Available at: https://github.com/frank-Regal/kinova_ar_teleop (Accessed: 15 April 2026).

[24] European Commission. Directorate-General for Research and Innovation (2021) *Industry 5.0: Towards a sustainable, human-centric and resilient European industry*. Publications Office of the European Union. Available at: https://doi.org/10.2777/308407

Authorized licensed use limited to: University of Washington Libraries. Downloaded on October 10,2026 at 16:34:36 UTC from IEEE Xplore. Restrictions apply.